PROYECTO 1

Objetivos generales

- Familiarizar al estudiante con el lenguaje JAVA de escritorio con SWING.
- Familiarizar al estudiante con el proceso básico de un analizador léxico.
- **Elaborar la lógica para la solución del problema planteado.**

Objectivos específicos

- Conocer las diferencias de sintaxis entre los lenguajes que el estudiante usa.
- Implementar algoritmos para el reconocimiento de patrones y recuperación de errores.
- Aplicar buenas prácticas de programación.

Procedimiento teórico-práctico a realizar

Creación de la expresión regular que describa el patrón de cada token.
Gramática regular de cada token.
AFD de cada token.

- Creación del AFD que acepte todos los tokens
- Diagrama de transiciones del AFD
- Tabla de transiciones del AFD

Descripción de la actividad

Una de las tareas de un analizador léxico es la detección y reconocimiento de tokens válidos para el lenguaje en cuestión,así como el reconocimiento de errores de tokens no definidos o caracteres que no se encuentran en el alfabeto del lenguaje. El analizador léxico también debe reaccionar correctamente ante los posibles errores que se encuentren, indicando la posición del error y una descripción del mismo.
La siguiente práctica invita al estudiante a idear algoritmos por medio de un autómata que permitan reconocer patrones en una cadena de caracteres y reportar errores que se presenten a lo largo del análisis.

Editor

Carga de archivo de entrada

El usuario tiene la disponibilidad de cargar un archivo de entrada con el texto que desea analizar. El texto del archivo de texto debe ser cargado en un componente que permita su edición.

Edición de texto de entrada

El usuario tiene la disponibilidad de crear texto de entrada en un componente para su análisis. Este componente nos debe mostrar en qué fila y columna se encuentra el puntero de texto.

Búsqueda de patrones

El usuario tiene la disponibilidad de indicar una palabra que será buscada dentro del texto de entrada y al presionar un botón, una nueva área mostrará el texto de entrada resaltando el texto que coincida con la cadena buscada.

Guardar cambios del texto de entrada

El usuario deberá tener la opción de exportar el texto de entrada en un archivo de texto.

Análisis de tokens

Para esta practica, el alfabeto permitido está compuesto por los siguientes símbolos:

- Letras de la ‘a’ a la ‘z’, ya sea mayúsculas o minúsculas.
- Dígitos del 0 al 9
- Signos de puntuación: punto (.), coma (,), punto y coma (;), dos puntos (:)
- Operadores aritméticos: Suma (+), Resta (-), Multiplicación (*), división (/), módulo (%),igual (=)
- Signos de agrupación: Paréntesis derecho ‘ (‘, Paréntesis izquierdo, ‘) ‘, Corchete izquierdo ‘ [‘, corchete derecho ‘] ‘, llave izquierda ‘ { ‘, llave derecha ‘ } ‘
- Espacio, salto de línea, Tabulaciones

Cualquier carácter que no esté definido acá deberá ser detectado como un error.

Los tokens válidos se describen a continuación:

- **Identificador**: Son las palabras que cumplen con iniciar con una letra y pueden estar seguidas de muchas letras o muchos dígitos.
- **Número**: Son palabras que cumplen con tener al menos un dígito o más, y solo pueden contener dígitos.
- **Decimal**: Son palabras que cumplen con tener al menos un dígito o más, seguido de un punto, seguido de uno o más dígitos.
- **Cadena:** Palabras que empiecen con comilla seguido de cualquier símbolo permitido seguido de comilla Ejemplo: “ HOLA MUNDO ”
- **Palabras Reservadas**: Son secuencias de caracteres formadas por letras (mayúsculas o minúsculas) que tienen un **significado especial dentro del lenguaje** y **no pueden ser utilizadas como identificadores**. En esta práctica, se reconocen las siguientes palabras reservadas:
    - SI,si
    - ENTONCES,entonces
    - PARA,para
    - ESCRIBIR,escribir

- **Puntuación**: Ser alguno de los signos de puntuación
- **Operador**: Ser alguno de los operadores aritméticos
- **Agrupacion**: Ser alguno de los signos de agrupación
- **Comentarios de línea:** Comentarios de línea **Ejemplo:** // adsfasdfasd
- **Comentarios de bloque** Ejemplo: /* Comentarios */
  Los comentarios se ignoran durante el análisis.

NOTA: los espacios y los saltos de línea indican la separación de palabras, pero no tienen otro significado en particular.

Colores para cada tipo de token

Mientras el usuario va escribiendo en el área de entrada, deberá de detectar automáticamente el tipo de token y **pintarlas con un color diferente** según el tipo de token:

- Azul: Palabras reservadas
- Cafés: identificadores
- Verde: Números
- Negro: Decimales
- Verde Oscuro: Comentarios de Línea:
- Verde Oscuro: Comentarios de bloque:
- Amarillo: Operador
- Morado: Agrupación
- Rojo: Errores

Dado lo anterior, el usuario tiene la disponibilidad de ejecutar un análisis sobre el texto de entrada, esta funcionalidad genera los siguientes reportes:

Reporte de errores

Si existieran errores, se debe generar una tabla con la siguiente información

- Símbolo o cadena de error
- Posición (fila y columna dentro del área de texto)
- Descripción (se tendrá que especificar qué carácter era lo que se esperaba)

Ejemplo:
(20.a,Fila 1 Columna 2, se esperaba un número)

Reporte de tokens

En este reporte se listan todos los tokens con la siguiente información:

- Nombre del Token
- Lexema
- Posición (fila y columna dentro del área de texto)

El usuario también debe contar con la funcionalidad de recuento de lexemas, la cual se debe habilitar cuando no existan errores en el texto de entrada, y consta de una tabla con la siguiente información:

- Lexema
- Tipo de Token (número, identificador, decimal, agrupación, etc)
- Cantidad de veces que aparece en el texto de entrada.

Generación del autómata

El uso de un autómata finito determinista en su forma formal es muy importante para poder optimizar código y hacerlo escalable por lo tanto durante este proyecto se considera obligatorio la implementación de un autómata finito determinista, de esta manera sería muy fácil el configurar una nueva gramática únicamente modificando sus reglas gramaticales o funciones de transición.

Así también es necesario indicar los movimientos que realiza este autómata por medio de su tabla de estados cuando se está analizando un token. Se tendrá una consola donde se impriman los movimientos del autómata Ejemplo:

“Me moví del estado A al estado B con un carácter J ”.

Otro Ejemplo:

Con la palabra: hola

Me moví del estado 1 al estado 2 con una h,
Me moví del estado 2 al estado 3 con una o
Me moví del estado 3 al estado 4 con una l
Me moví del estado 4 al estado 5 con una a
Guardando token Identificador Lexema: hola.
Reiniciando Automata …

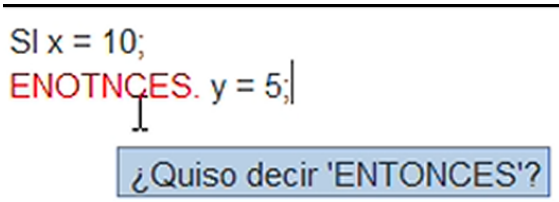
Recuperación de errores

La forma de recuperarse de un error es enviando el token de error y a su vez reiniciando el análisis para continuar. Entonces cuando exista un error se tendrá que seguir evaluando buscando un posible siguiente error o bien un token válido, ejemplo:

585f3.40 <- Error, en 585f, pero detecta token decimal = 3.40

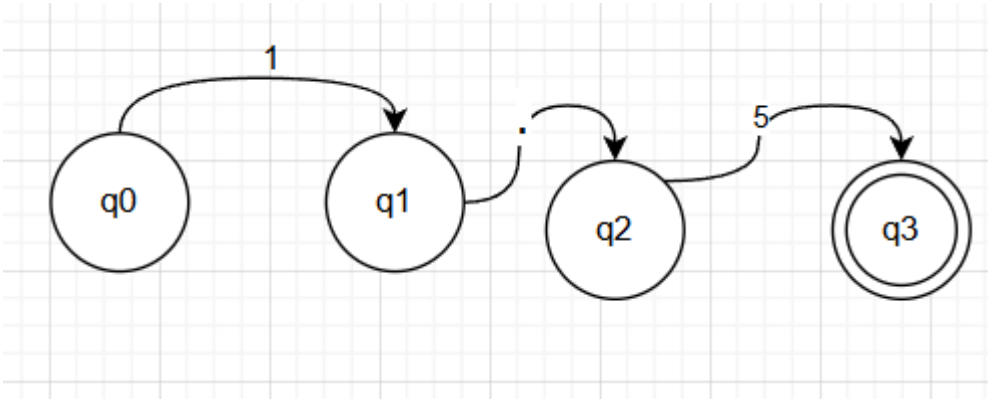
Integración de recuperación de errores avanzada

Al momento de que el autómata detecte un error esta cadena sera pintada de color rojo como se describe arriba, se tendrá la opcion de que al momento de pasar el cursor sobre el error, se muestre una sugerencias de corrección : por ejemplo, si escriben ENOTNCES. , sugerir ENTONCES (esto para las palabras reservadas) Ejemplo:



Generación visual del diagrama de transiciones.

Al generar el reporte de tokens por cada token reconocido el usuario también tendrá la opción de generar los movimientos que realizó el autómata visualmente.
Ejemplo con el token decimal con lexema 1.5 :



Simulación paso a paso

El analizador léxico deberá contar con un **modo depuración** en el cual el usuario pueda ejecutar el análisis de manera **paso a paso** para observar cómo el autómata procesa una cadena de entrada.

En este modo, el usuario ingresará una palabra o fragmento de texto y el sistema deberá mostrar:

1. **Estado inicial** en el que comienza el autómata.
2. **Lectura de cada carácter** de la palabra, indicando:
    - El carácter leído.
    - El estado actual.
    - El estado al que se transita con dicho carácter.
3. **Construcción del lexema**, mostrando cómo se va formando carácter a carácter.
4. **Aceptación del token**, cuando se llegue a un estado de aceptación y se determine el tipo de token reconocido.
5. **Detección de errores**, si en algún momento no existe transición posible con el carácter leído.

Cada paso debe mostrarse claramente al usuario.

El modo depuración debe permitir al usuario:

- **Avanzar paso a paso** (consumir un carácter a la vez).
- **Regresar un paso**
- **Ejecutar hasta finalizar el token actual**.
- **Reiniciar el análisis** para comenzar de nuevo desde el estado inicial.

Importante

- Práctica obligatoria para tener derecho a la siguiente práctica o proyecto
- Es válido utilizar algún IDE o cualquier editor de texto.
- Las copias obtendrán nota de cero y se notificará a coordinación.
- Si se va a utilizar código de internet, entender la funcionalidad para que se tome como válido.
- No se permite usar ninguna función propia de java o librería que tenga que ver con reconocimiento de patrones o manejo directo de cadenas. Solo manejo de chars.
- Es obligatorio la implementación del autómata para el análisis del texto.

Entrega

La fecha de entrega es el dia 2 de octubre antes de las 16:30 horas por medio de la plataforma classroom, los componentes a entregar en repositorio git son:

- Código fuente
- Trabajo Teórico-Práctico
- Manual de usuario y técnico

Calificación

Pendiente